

Home > Tutorials > Artificial Intelligence (AI)

Using GPT-3.5 and GPT-4 via the OpenAI API in Python

In this tutorial, you'll learn how to work with the OpenAI Python package to programmatically have conversations with ChatGPT.

Apr 2023 · 14 min read



Richie Cotton
Webinar & podcast host, course and book author, spends all day chit-chatting about data

TOPICS

Artificial Intelligence (AI)

Python

ChatGPT is a cutting-edge large language model for generating text. It's already changing how we write almost every type of text, from tutorials like this one, to auto-generated product descriptions, [Bing's search engine results](#), and dozens of data use cases as described in the [ChatGPT for Data Science cheat sheet](#).

For interactive use, the [web interface to ChatGPT](#) is ideal. However, OpenAI (the company behind ChatGPT) also has an application programming interface (API) that lets you interact with ChatGPT and their other models using code.

In this tutorial, you'll learn how to work with the `openai` Python package to programmatically have conversations with ChatGPT.

Note that OpenAI charges to use the GPT API. (Free credits are sometimes provided to new users, but who gets credit and how long this deal will last is not transparent.) It costs \$0.002 / 1000 tokens, where 1000 tokens equal about 750 words. Running all the examples in this tutorial once should cost less than 2 US cents (but if you rerun tasks, you will be charged every time).

When Should You Use the API Rather Than The Web Interface?

The ChatGPT web application is a great interface to GPT models. However, if you want to include AI into a data pipeline or into software, the API is more appropriate. Some possible use cases for data practitioners include:

- Pulling in data from a database or another API, then asking GPT to summarize it or generate a report about it
- Embedding GPT functionality in a dashboard to automatically provide a text summary of the results
- Providing a natural language interface to your data mart
- Performing research by pulling in journal papers through the scholarly ([PyPI](#), [Conda](#)) package, and getting GPT to summarize the results

Setup An OpenAI Developer Account

To use the API, you need to create a developer account with OpenAI. You'll need to have your email address, phone number, and debit or credit card details handy.

Follow these steps:

1. Go to the [API signup page](#).
2. Create your account (you'll need to provide your email address and your phone number).
3. Go to the [API keys page](#).
4. Create a new secret key.
5. Take a copy of this key. (If you lose it, delete the key and create a new one.)
6. Go to the [Payment Methods page](#).
7. Click 'Add payment method' and fill in your card details.

Securely Store Your Account Credentials

The secret key needs to be kept secret! Otherwise, other people can use it to access the API, and you will pay for it. The following steps describe how to securely store your key using DataCamp Workspace. If you are using a different platform, please check the documentation for that platform. You can also ask ChatGPT for advice. Here's a suggested prompt:

> You are an IT security expert. You are speaking to a data scientist. Explain the best practices for securely storing private keys used for API access.

1. In your workspace, click on Integrations
2. Click on the "Create integration" plus button
3. Select an "Environment Variables" integration
4. In the "Name" field, type "OPENAI". In the "Value" field, paste in your secret key
5. Click "Create", and connect the new integration

Setting up Python

To use GPT via the API, you need to import the `os` and `openai` Python packages.

If you are using a Jupyter Notebook (like DataCamp Workspace), it's also helpful to import some functions from `IPython.display`.

One example also uses the `yfinance` package to retrieve stock prices.

```
# Import the os package
import os

# Import the openai package
import openai

# From the IPython.display package, import display and Markdown
from IPython.display import display, Markdown

# Import yfinance as yf
import yfinance as yf
```

🔗 Explain code



OpenAI

The other setup task is to put the environment variable you just created in a place that the `openai` package can see it.

```
# Set openai.api_key to the OPENAI environment variable
openai.api_key = os.environ["OPENAI"]
```

🔗 Explain code



OpenAI

The Code Pattern for Calling GPT via the API

The code pattern to call the OpenAI API and get a chat response is as follows:



```
response = openai.ChatCompletion.create(
    model="MODEL_NAME",
    messages=[{"role": "system", "content": 'SPECIFY HOW THE AI ASSISTANT SHOULD BEHAVE' },
              {"role": "user", "content": 'SPECIFY WANT YOU WANT THE AI TO DO' }])
```

🔗 Explain code



Buy Now >

OpenAI API model names for GPT

The model names are listed in the [Model Overview](#) page of the developer documentation. In this tutorial, you'll be using `gpt-3.5-turbo`, which is the latest model used by ChatGPT that has public API access. (When it becomes broadly available, you'll want to switch to `gpt-4`.)

OpenAI API GPT message types

There are three types of message documented in the [Introduction to the Chat](#) documentation:

- `system` messages describe the behavior of the AI assistant. A useful system message for data science use cases is "You are a helpful assistant who understands data science."
- `user` messages describe what you want the AI assistant to say. We'll cover examples of user messages throughout this tutorial
- `assistant` messages describe previous responses in the conversation. We'll cover how to have an interactive conversation in later tasks

The first message should be a system message. Additional messages should alternate between the user and the assistant.

Your First Conversation: Generating a Dataset

Generating sample datasets is useful for testing your code against different data scenarios or for demonstrating code to others. To get a useful response from GPT, you need to be precise and specify the details of your dataset, including:

- the number of rows and columns
- the names of the columns
- a description of what each column contains
- the output format of the dataset

Here's an example user message to create a dataset.

```
Create a small dataset about total sales over the last year.
The format of the dataset should be a data frame with 12 rows and 2 columns.
The columns should be called "month" and "total_sales_usd".
The "month" column should contain the shortened forms of month names
from "Jan" to "Dec". The "total_sales_usd" column should
contain random numeric values taken from a normal distribution
with mean 100000 and standard deviation 5000. Provide Python code to
generate the dataset, then provide the output in the format of a markdown table.
```

🔗 Explain code



Let's include this message in the previous code pattern.

```
# Define the system message
system_msg = 'You are a helpful assistant who understands data science.'

# Define the user message
user_msg = 'Create a small dataset about total sales over the last year. The form

# Create a dataset using GPT
response = openai.ChatCompletion.create(model="gpt-3.5-turbo",
                                       messages=[{"role": "system", "content": s
                                                {"role": "user", "content": user_msg}]])
```

🔗 Explain code



Check the response from GPT is OK

API calls are "risky" because problems can occur outside of your notebook, like internet connectivity issues, or a problem with the server sending you data, or because you ran out of API credit. You should check that the response you get is OK.

GPT models return a status code with one of four values, documented in the [Response format](#) section of the Chat documentation.

- `stop`: API returned complete model output
- `length`: Incomplete model output due to `max_tokens` parameter or token limit
- `content_filter`: Omitted content due to a flag from our content filters
- `null`: API response still in progress or incomplete

The GPT API sends data to Python in JSON format, so the response variable contains deeply nested lists and dictionaries. It's a bit of a pain to work with!

For a response variable named `response`, the status code is stored in the following place.

```
response["choices"][0]["finish_reason"]
```

🔗 Explain code



Extract the AI assistant's message

Buried within the response variable is the text we asked GPT to generate. Luckily, it's always in the same place.

```
response["choices"][0]["message"]["content"]
```

🔗 Explain code



The response content can be printed as usual with `print(content)`, but it's Markdown content, which Jupyter notebooks can render, via `display(Markdown(content))`

Here's the Python code to generate the dataset:

```
import numpy as np
import pandas as pd
# Set random seed for reproducibility
np.random.seed(42)
# Generate random sales data
sales_data = np.random.normal(loc=100000, scale=5000, size=12)
# Create month abbreviation list
month_abbr = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
# Create dataframe
sales_df = pd.DataFrame({'month': month_abbr, 'total_sales_usd': sales_data})
# Print dataframe
print(sales_df)
```

And here's the output in markdown format:

```
| month | total_sales_usd |
|-----|-----|
| Jan   | 98728.961189    |
| Feb   | 106931.030292   |
| Mar   | 101599.514152   |
```

Apr	97644.534384	
May	103013.191014	
Jun	102781.514665	
Jul	100157.741173	
Aug	104849.281004	
Sep	100007.081991	
Oct	94080.272682	
Nov	96240.993328	
Dec	104719.371461	

🔗 Explain code

OpenAI

Use a Helper Function to call GPT

You need to write a lot of repetitive boilerplate code to do these three simple things. Having a wrapper function to abstract away the boring bits is useful. That way, we can focus on data science use cases.

Hopefully, OpenAI will improve the interface to their Python package so this sort of thing is built-in. In the meantime, feel free to use this in your own code.

The function takes two arguments.

- `system`: A string containing the system message.
- `user_assistant`: An array of strings that alternate user message then assistant message.

The return value is the generated content.

```
def chat(system, user_assistant):  
    assert isinstance(system, str), "`system` should be a string"  
    assert isinstance(user_assistant, list), "`user_assistant` should be a list"  
    system_msg = [{"role": "system", "content": system}]  
    user_assistant_msgs = [  
        {"role": "assistant", "content": user_assistant[i]} if i % 2 else {"role":  
            "user", "content": user_assistant[i]} for i in range(len(user_assistant))  
    ]  
  
    msgs = system_msg + user_assistant_msgs  
    response = openai.ChatCompletion.create(model="gpt-3.5-turbo",  
                                           messages=msgs)  
    status_code = response["choices"][0]["finish_reason"]  
    assert status_code == "stop", f"The status code was {status_code}."  
    return response["choices"][0]["message"]["content"]
```

🔗 Explain code

OpenAI

Example usage of this function is

```
response_fn_test = chat("You are a machine learning expert.", ["Explain what  
display(Markdown(response_fn_test))
```

🔗 Explain code

OpenAI

A neural network is a type of machine learning model that is inspired by the structure and behavior of a neural network can be adjusted by changing the weights. Neural networks are often used for tasks such as image classification, speech recognition, and natural language processing.

🔗 Explain code

OpenAI

Reusing Responses from the AI Assistant

In many situations, you will wish to form a longer conversation with the AI. That is, you send a prompt to GPT, get a response back, and send another prompt to continue the chat. In this case, you need to include the previous response from GPT in the second call to the API, so that GPT has the full context. This will improve the accuracy of the response and increase consistency across the conversation.

In order to reuse GPT's message, you retrieve it from the response, and then pass it into a new call to chat.

Example: Analyzing the sample dataset

Let's try calculating the mean of the sales column from the dataset that was previously generated. Note that because we didn't use the `chat()` function the first time, we have to use the longer subsetting code to get at the previous response text. If you use `chat()`, the code is simpler.

```
# Assign the content from the response in Task 1 to assistant_msg  
assistant_msg = response["choices"][0]["message"]["content"]  
  
# Define a new user message  
user_msg2 = 'Using the dataset you just created, write code to calculate the mean  
sales over the last year.'  
  
# Create an array of user and assistant messages  
user_assistant_msgs = [user_msg, assistant_msg, user_msg2]  
  
# Get GPT to perform the request  
response_calc = chat(system_msg, user_assistant_msgs)  
  
# Display the generated content  
display(Markdown(response_calc))
```

🔗 Explain code

OpenAI

Sure! Here's the code to calculate the mean of the `total_sales_usd` column:

```
python  
mean_sales = sales_df['total_sales_usd'].mean()  
print("Mean sales: $", round(mean_sales, 2))  
...
```

And here's the output of this code:

```
...  
Mean sales: $ 100077.57  
...
```

Therefore, the mean of total sales over the last year is about \$100,077.57.

🔗 Explain code

OpenAI

Using GPT in a Pipeline

A huge advantage of using the API over the web interface is that you can combine GPT with other APIs. Pulling in data from one or more sources, then applying AI to it is a powerful workflow.

Applying GPT AI to weather data

Here, we'll pull in a weather forecast with the `weather2` package (PyPI) and use GPT to come up with ideas for activities.

```
# Import the weather2 package
import weather

# Get the forecast for Miami
location = "Miami"
forecast = weather.forecast(location)

# Pull out forecast data for midday tomorrow
fcast = forecast.tomorrow["12:00"]

# Create a prompt
user_msg_weather = f"In {location} at midday tomorrow, the temperature is forecas

# Call GPT
response_activities = chat("You are a travel guide.", [user_msg_weather])

display(Markdown(response_activities))
```

🔗 Explain code

OpenAI

With mild temperatures and calm winds, Miami is the perfect place for leisur ct

1. Visit Miami's beaches and soak up some sun or take a dip in the ocean!
2. Explore Miami's art scene with a visit to the Perez Art Museum Miami or the Wy
3. Take a stroll along the famous Ocean Drive and enjoy the colorful Art Deco arc
4. Head to Little Havana to experience the Cuban culture and delicious cuisine.
5. Enjoy a scenic walk or bike ride through one of Miami's many parks, such as Ba
6. Visit the Miami Seaquarium and see some incredible marine life up close.
7. Take a boat tour to see the stunning Miami skyline from the water.
8. Shopping enthusiasts can explore the many high-end boutiques and outdoor shopp
9. Foodies can venture to one of the many food festivals happening throughout the
10. Finally, there are plenty of nightclubs and live music venues to keep the nig

🔗 Explain code

OpenAI

Nice! Part of me wishes I was in Miami tomorrow!

Take it to the Next Level

You can learn more about using GPT in the [Introduction to ChatGPT](#) course. (A full course on working with the OpenAI API is coming soon!)

For a reference of what you just learned, take a copy of the [OpenAI API in Python Cheat Sheet](#) or watch the [Getting Started with the OpenAI API and ChatGPT](#) live training recording.

If you are interested in text-to-speech transcription, read the [Converting Speech to Text with the OpenAI Whisper API](#) tutorial.

- TOPICS
- Artificial Intelligence (AI)

Python

Top Python Courses

Introduction to Python

Beginner 4 hr 4.9M

Master the basics of data analysis with Python in just four hours. This online course will introduce the Python interface and explore popular packages.

See Details →

Start Course

See More →

Related

- What is Synthetic Data?

Abid Ali Awan

How to Learn Python From Scratch in 2023: An Expert...

Matt Crabtree

Generative AI in eCommerce: It Started With a Prompt

Benji Taylor

See More →

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.



LEARN

- Learn Python

Learn R

Learn AI

Learn SQL

Learn Power BI

Learn Tableau

Assessments

Career Tracks

Skill Tracks

Courses

Data Science Roadmap

DATA COURSES

- Upcoming Courses
- Python Courses
- R Courses
- SQL Courses
- Power BI Courses
- Tableau Courses
- Spreadsheets Courses
- Data Analysis Courses
- Data Visualization Courses
- Machine Learning Courses
- Data Engineering Courses

WORKSPACE

- Get Started
- Templates
- Integrations
- Documentation

CERTIFICATION

- Certifications
- Data Scientist
- Data Analyst
- Data Engineer
- Hire Data Professionals

RESOURCES

- Resource Center
- Upcoming Events
- Blog
- Tutorials
- Open Source
- RDocumentation
- Course Editor
- Book a Demo with DataCamp for Business
- Data Portfolio
- Portfolio Leaderboard

PLANS

- Pricing
- For Business
- For Universities
- Discounts, Promos & Sales
- DataCamp Donates

SUPPORT

- Help Center
- Become an Instructor
- Become an Affiliate

ABOUT

- About Us
- Learner Stories
- Careers
- Press
- Leadership
- Contact Us

